

定日镜场的工作参数计算及优化设计

摘要

本文针对给定的定日镜场布局给出了计算关键参数的方法，以及在考虑多种决策变量的情况下，给出了对于优化定日镜场布局的思考。

针对问题一：对于给定定日镜场布局的关键工作参数计算，我们首先通过所在地的经纬度和给定的时间，可直接计算太阳的高度角和方位角。通过不同坐标系下坐标的转换，能够将太阳的高度角和方位角转化为镜场坐标系下的向量，从而将光束的传播问题转换为镜场坐标系下向量和坐标的计算。对于截断损失和阴影遮挡损失的计算，分别采用了**蒙特卡洛法**和**光束追踪法**对其进行近似估计，对于大气透射率和余弦效率，则通过**向量和坐标进行直接计算**。在上述准则的计算下，我们得到问题一中布局的年平均光学效率为 **0.6393**，年平均输出热功率为 **39.0112MW**，单位面积镜面年平均输出热功率为 **0.6210kW/m²**，其它相关关键参数请见附表。

针对问题二：在**多目标优化**的定日镜场布局问题中，由于集热器坐标的改变引起的计算时间复杂度过大，同时集热器坐标对镜场总体热功率影响较大，我们首先考虑缩小集热器的坐标范围。由于定日镜场的**对称性**，我们首先遍历了在定日镜均匀分布的定日镜场内集热器的坐标对应的镜场总体热功率，获得了吸收塔的较优位置。而后基于**Campo 分布**获取围绕较优集热器坐标的定日镜布局。在此基础上通过**模拟退火算法**，实现对定日镜尺寸、安装的高度、吸收塔的坐标进行扰动进行全局最优解的搜索。同时扰动会使得部分情况下定日镜的安装高度增大，有效弥补了 Campo 分布其它效率较优但阴影遮挡效率较低的缺陷。在上述准则的指导下的布局，相较问题一的布局，单位面积镜面的年平均输出提升了约 10%。在满足年平均输出热功率为 **60.7191MW** 的基础上，单位面积镜面的年平均输出达到 **0.6968kW/m²**，年平均光学效率为 **0.7197**。

针对问题三：问题三在问题二的基础上进一步增添了决策变量：定日镜间不同的尺寸和不同的安装高度。基于问题二的定日镜场的分布，我们使用了**基于贪婪算法的膨胀—收缩算法**优化不同尺寸间定日镜的布局。同时我们**类比于向日葵在山坡上的生长**，即可通过增加阴影遮挡效率低下区域的安装高度从而提升阴影遮挡效率低下区域的定光镜的光学效率。同时对于定日镜的安装高度，同样采用**模拟退火算法**进行扰动，寻找更多变量下的较优解。在上述准则指导下的布局，相较问题二的布局进行了进一步优化，在满足年平均输出热功率为 **60.5728MW** 的基础上，单位面积镜面的年平均输出达到了 **0.6998kW/m²**，年平均光学效率达到了 **0.7239**。

本文对于问题的思考重点在于空间坐标系下的计算、复杂物理量的近似和对决策变量庞大的模型的相应优化。在文章的最后，也针对对被忽略的一些因素进行了总结和思考，分析了模型的优缺点。

关键词：塔式光热电站定日镜场 蒙特卡洛法 光束追踪法 模拟退火算法 膨胀—收缩算法

一、问题重述

问题背景：构建以新能源为主体的新型电力系统，是我国实现“碳达峰”、“碳中和”目标的一项重要措施。塔式太阳能光热发电正是一种低碳环保的新型清洁能源技术，对其的研究具有重要的意义。本题给出塔式电站的布局 and 关键装置的介绍，要求计算其工作时的关键参数和寻求给定约束下的最优化布局。

问题一：给定所有定日镜和吸收塔的位置坐标、尺寸、高度，在考虑多种因素的情况下计算给定布局的效率和热功率

问题二：定日镜场的额定年平均输出热功率为 60 MW。所有定日镜尺寸及安装高度相同。在输出功率和定日镜尺寸及安装高度的约束下，设计定日镜场的布局，使得定日镜场在达到额定功率的条件下，单位镜面面积年平均输出热功率尽量大。

问题三：问题三在问题二的基础上取消了所有定日镜尺寸及安装高度相同的限制，重新寻求最优的定日镜场布局参数。

二、问题分析

问题一的分析：

首先通过给定的经纬度以及具体时间可以计算定日镜场在特定时刻的太阳高度角 θ_s 和太阳方位角 γ_s ，并通过太阳高度角和太阳方位角将太阳光方向转换为镜面坐标系下的向量。

由太阳中心点发出的光线经定日镜中心反射后指向集热器中心可知，对于定日镜中心确定的定日镜，其反射光线方向确定。由入射光线和反射光线可以确定定日镜面的法向量方向。

定日镜的光学效率是阴影遮挡效率、余弦效率、大气透射率、集热器截断效率以及镜面反射率之积。计算阴影遮挡效率可先将定日镜进行分割处理，而后由**蒙特卡洛算法**模拟获得；余弦损失可由太阳光方向的单位向量与定日镜面法向单位向量作点积计算；大气透射率可由距离公式计算距离后代入公式计算获得；截断效率可由**光线追迹法**进行近似；镜面反射率可视为常数。

计算出每个定日镜的光学效率后，年平均输出热功率与年平均输出热功率均可依据相关公式计算得出。

问题二的分析：

问题二为典型的多目标优化问题，且决策变量较多。首先考虑缩小吸收塔的坐标范围。不仅是由于吸收塔对整体热效率影响较大，也因为对吸收塔坐标的扰动范围增大会极大增加程序的时间复杂度。由于定日镜场的**对称性**，我们先求出了在定日镜分布均匀的镜场中吸收塔的最佳坐标。接着通过 **Campo 方式**对定日镜进行排布，而后将定日镜的尺寸、安装的高度、吸收塔的坐标进行扰动，运用**模拟退火算法**寻求较优的布局。

问题三的分析：

问题三在问题二的基础上，定日镜尺寸和安装高度不再统一。因为 E. Carrizosa 等人的论文中提及，在输出相同的功率的情况下，小定日镜所需的受光面积较大定日镜所需的受光面积更大。所以先对于固定的安装高度采用**基于贪婪算法的膨胀——收缩算法**优化不同尺寸的定日镜的位置，然后运用创新性的理论分析结合**模拟退火算法**扰动安装高度，最终获取较优的布局。

三、模型假设

假设一：整个半径为 350m 的定日镜场内地面平坦，海拔一致。

假设二：由于定日镜场位于甘肃酒泉附近，海拔高，降水少，于是忽略太阳光进入大气时的折射。即认为太阳散发的“锥形光”沿直线入射到镜面。

假设三：经过计算（计算过程体现在第五部分“模型的求解”部分），锥形光束的光锥锥角大约为 9.3mrad，已经是一个需要考虑的大小。但是，由于太阳相对地球而言十分庞大，我们在计算时不再考虑入射的锥形光束之间的角度，仅考虑光锥锥角。

假设四：所有定日镜的镜面反射率均为 0.92，且由于定期清理，不会发生变化。

假设五：假设阴影遮挡时的太阳光为光线，忽略光锥。

假设六：假设一年 365 天整，不考虑每年多出的 1/4 天，即 3 月 20 日对应 D=324.

假设七：假设每个月的 21 日全天为晴。第七部分“模型的总结”中会考虑阴天造成的影响，并对算法进行改进。

四、符号说明

符号	说明	单位
∂_s	太阳高度角	rad
γ_s	太阳方位角	rad
φ	当地纬度 (+)	度
ω	太阳时角	rad
ST	当地时间	时间单位
δ	太阳赤纬角	rad
D	以春分为第零天的天数	天
DNI	法向直接辐射辐照度	kW / m^2
E_{field}	热输出功率	kW
η_{sb}	阴影遮挡效率	1
η_{cos}	余弦效率	1
η_{at}	大气透射率	1
η_{trunc}	集热器截断效率	1
η_{ref}	镜面反射率	1

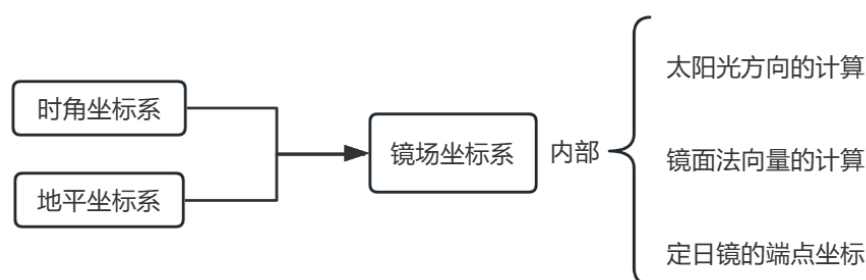
d_{HR}	镜面中心到集热器中心的距离	m
E_{rated}	额定热功率	MW

五、模型建立与求解

5.1 问题一模型的建立与求解

5.1.1 模型的建立

5.1.1.1 坐标系的建立



1. 时角坐标系和地平坐标系的概念

经查阅资料，本题中定日镜采用的是视日轨跟踪方式，即根据电站的建设位置提前计算好太阳的运行轨迹，并通过微型计算机直接控制定日镜的俯仰角和方位角来调整定日镜的运行姿态。这种跟踪方式不受外部环境的影响，具有较高的可靠性。本题已经知道了吸收塔的位置，定日镜的尺寸、高度，以及定日镜中心的位置。想要计算光学效率与热功率，这样就需要对太阳位置与定日镜反射光这一过程进行建模。

首先建立时角坐标系，时角坐标系是以地心为原点，通过时角 ω 与赤纬角 δ ，我们可以轻松对天体表面的点进行定位，也可以更直观的了解太阳高度角与经纬度的宏观概念。其中，太阳赤纬角 δ 的计算公式为

$$\delta = \arcsin[\sin(2\pi D / 365) * \sin(2\pi * 23.45 / 360)] \quad (1)$$

太阳时角 ω 的计算公式为

$$\omega = (\pi / 12) * (ST - 12) \quad (2)$$

D 为以春分作为第 0 天起算的天数，ST 为当地时间。

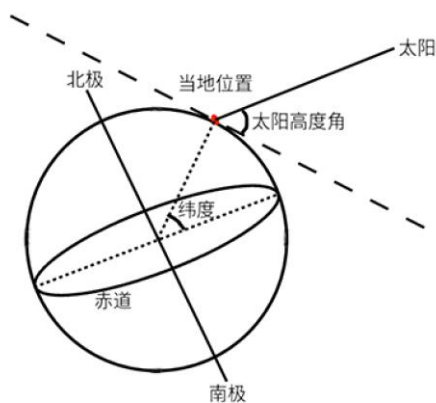
我们将时角坐标系中的“当地位置”放大，便成了当地的地平坐标系，地平坐标系是以观测者为原点，以地平面上的正东方为 x 轴正方向，正北方为 y 轴正方向，垂直于地面向上方向为 z 轴正方向。其中，太阳高度角 ∂_s 的计算公式为

$$\partial_s = \arcsin(\cos \delta * \cos \varphi * \cos \omega + \sin \delta * \sin \varphi) \quad (3)$$

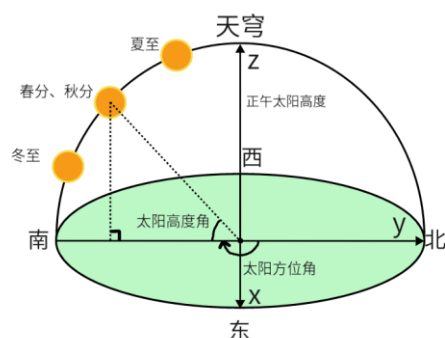
太阳方位角 γ_s 的计算公式为

$$\gamma_s = \arccos[(\sin \delta - \sin \partial_s * \sin \varphi) / (\cos \partial_s * \cos \varphi)] \quad (4)$$

φ 为当地纬度，北纬为正。同时，图中还画出了春分、夏至、秋分、冬至的正午太阳方位，看图可知，由于定日镜场位于北回归线以北，所以正午太阳的投影均位于观测者的正南方。



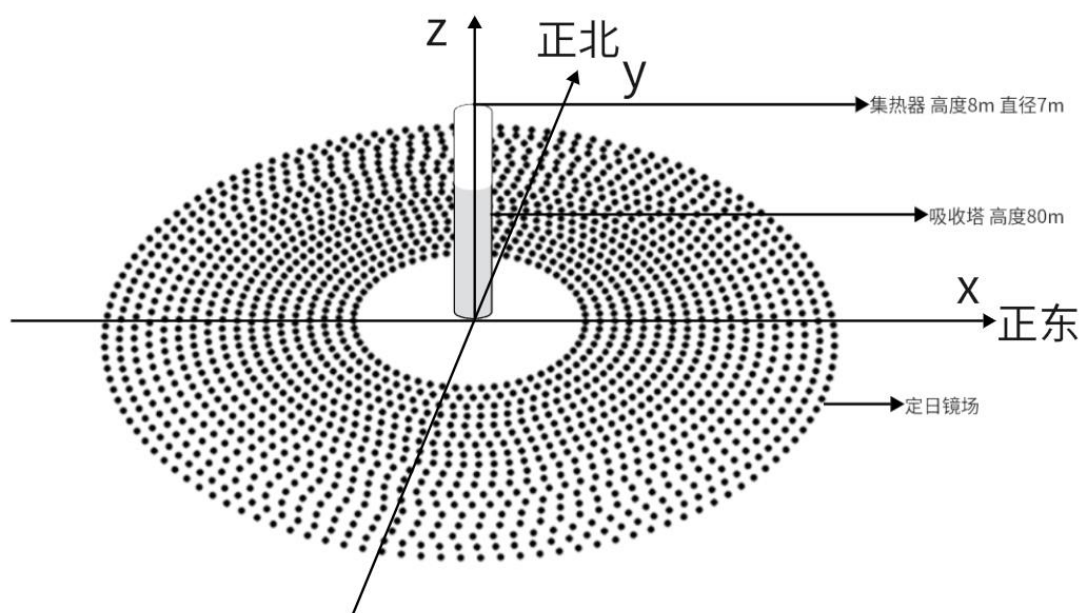
时角坐标系



地平坐标系

2. 镜场坐标系的建立

之后，我们再建立镜场坐标系，以圆形区域中心为原点，第一间中即为集热器底部，然后以正东为 x 轴正向，正北为 y 轴正向，垂直于地面向上方向为 z 轴正向建立镜场坐标系。



镜场坐标系

3. 在镜场坐标系中计算太阳光的方向：

我们将 D 和 ST 代入公式 (1) 计算出太阳赤纬角 δ ，将 ST 代入公式 (2) 计算出太阳时角 ω 。结合纬度 φ 可以由公式 (3)、(4) 计算得出太阳高度角 θ_s 和太阳方位角 γ_s 。通过太阳高度角和太阳方位角得到太阳光的方向向量，本质上就是球坐标向直角坐标的转换。故在镜场坐标系下太阳光的方向向量 $\vec{s}$ 满足关系（方向指向镜面）：

$$s_x = -\cos \partial_s \sin \gamma_s, s_y = -\cos \partial_s \cos \gamma_s, s_z = -\sin \partial_s \quad (5)$$

4.在镜场坐标系中计算定日镜镜面法向量:

已知定日镜中心的坐标和集热器中心坐标, 又因太阳中心点发出的光线经定日镜中心反射后指向集热器中心, 故反射光线的向量 $\vec{f}$ 表示为 (方向由定日镜中心指向集热器中心):

$$\left(x_{\text{集热器中心}} - x_{\text{定日镜中心}}, y_{\text{集热器中心}} - y_{\text{定日镜中心}}, z_{\text{集热器中心}} - z_{\text{定日镜中心}} \right)$$

由定日镜镜面法向量的方向为入射光线和反射光线的角平分线可知定日镜镜面法向量 $\vec{n}$ 满足向量加法:

$$\vec{n} = -\vec{s} + \vec{f} \quad (6)$$

5.在镜场坐标系中计算定日镜端点坐标:

由于定日镜的位置固定, 定日镜的法向量在该时刻方向也为一定值, 故定日镜的摆放方式也因此确定。此处使用定日镜的高度角及定日镜的方位角描述定日镜的摆放方式。

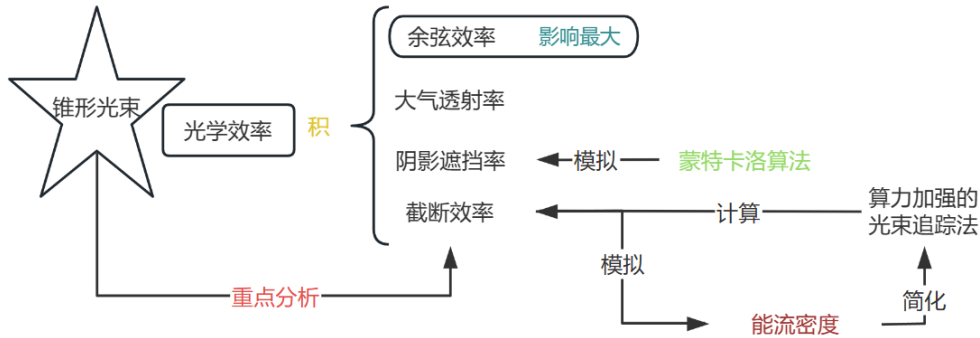
定日镜的高度角 θ_h 描述为定日镜平面与水平面的夹角。定日镜的方位角 θ_j 描述为定日镜的法向量在地面上的投影向量与正北方向 (即 y 轴) 的夹角。假设定日镜镜面单位法向量为 (n_x, n_y, n_z) , 则有关系式:

$$\begin{aligned} \cos \theta_h &= n_z, \sin \theta_h = \sqrt{1 - n_z^2} \quad (7) \\ \sin \theta_j &= -n_x / \sqrt{n_x^2 + n_y^2}, \cos \theta_j = -n_y / \sqrt{n_x^2 + n_y^2} \quad (8) \end{aligned}$$

进一步的, 已知定日镜的中心坐标 (x, y, z) 及其方位角和高度角的正余弦值, 已知其长度为 L , 宽度为 W , 则其端点 P 坐标满足:

$$\begin{aligned} P_1 &= \left(x - \frac{L}{2} \cos \theta_j - \frac{W}{2} \sin \theta_j, y - \frac{L}{2} \sin \theta_j + \frac{W}{2} \cos \theta_h \cos \theta_j, z - \frac{W}{2} \sin \theta_h \right) \\ P_2 &= \left(x - \frac{L}{2} \cos \theta_j + \frac{W}{2} \sin \theta_j, y - \frac{L}{2} \sin \theta_j - \frac{W}{2} \cos \theta_h \cos \theta_j, z + \frac{W}{2} \sin \theta_h \right) \\ P_3 &= \left(x + \frac{L}{2} \cos \theta_j + \frac{W}{2} \sin \theta_j, y + \frac{L}{2} \sin \theta_j - \frac{W}{2} \cos \theta_h \cos \theta_j, z + \frac{W}{2} \sin \theta_h \right) \quad (9) \\ P_4 &= \left(x + \frac{L}{2} \cos \theta_j - \frac{W}{2} \sin \theta_j, y + \frac{L}{2} \sin \theta_j + \frac{W}{2} \cos \theta_h \cos \theta_j, z - \frac{W}{2} \sin \theta_h \right) \end{aligned}$$

5.1.1.2 光学效率 η 的计算 ($\eta = \eta_{sb} * \eta_{cos} * \eta_{at} * \eta_{trunc} * \eta_{ref}$)



1.余弦效率的计算

定日镜需要将入射至定日镜中心的光线反射到集热器中心，因此定日镜需要旋转一定角度，使得入射光线与法线之间存在一个夹角 θ 。从而定日镜的实际采光面积小于定日镜的实际镜面面积。

假设入射光线的法向量为 $\vec{s}$ ，定日镜镜面的法向量为 $\vec{n}$ ，可知定日镜的余弦损失为：

$$\Delta = \frac{|\vec{n} \cdot \vec{s}|}{|\vec{n}||\vec{s}|}$$

定日镜的余弦效率即为：

$$\eta_{\cos} = 1 - \Delta = \frac{|\vec{n}||\vec{s}| - |\vec{n} \cdot \vec{s}|}{|\vec{n}||\vec{s}|} \quad (10)$$

2.大气透射率的计算

大气透射率是由于空气中的杂质等阻挡或因大气散射导致的能量耗散。题目中已给出计算大气透射率的公式：

$$\eta_{at} = 0.99321 - 0.0001176d_{HR} + 1.97 \times 10^{-8} \times d_{HR}^2 (d_{HR} \leq 1000) \quad (11)$$

其中 d_{HR} 为定日镜中心到集热器中心的距离（单位为 m）。由题知定日镜场半径为 350m，集热器高度为 84m。由两点间距离公式：

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2} \quad (12)$$

得极限情况定日镜中心距离与集热器中心距离均不会超过 1000m，因此通过两点坐标计算距离代入公式即可计算每面定日镜的大气透射率 η_{at} 。

3.阴影遮挡率的计算

首先考虑太阳光是锥形光束，定日镜中心距离通常在 10m 以内，产生的光圈半径 $r = d \tan \alpha$ 仅为 0.04m。此时可将锥形光束近似看作平行光，以便对阴影与遮挡效率的计算进行相应的简化。

在考虑光线的入射时，由于定日镜排布较密集，且高度角和方位角不同，可能会出现入射光线被其它定日镜遮挡的情况，即被遮挡的定日镜位于其它定日镜的阴影内。这部分光线损失称为阴影损失。

在考虑光线的反射时，由于光线的反射会改变光路的方向，可能会出现反射光线被其它定日镜遮挡的情况，即反射光线的路径中途径其它定日镜镜面。这部分光线损失称为遮挡损失。

由于阴影损失和遮挡损失均由入射和反射时定日镜间相互影响产生，两者本质相似，且计算方法类似，此处将阴影损失和遮挡损失运用蒙特卡洛方法并行计算。^[1]

蒙特卡洛算法的简单介绍：

蒙特卡洛方法是一种近似推断的方法，通过采样大量粒子的方法来求解期望、均值、面积、积分等问题。

具体到阴影遮挡效率的算法中，即通过采样大量镜面上的点，分析其入射及反射光线是否会由于阴影或遮挡而损失，最终统计总分析光线数量和损失光线数量，从而得到定日镜的阴影遮挡损失。

具体步骤如下：

a.假设：

由于定日镜长宽为有限值，其阴影和遮挡所能影响的范围有限，故假设仅在待分析定日镜周围 r 内的定日镜可能对待分析定日镜造成阴影遮挡的影响。

b.预处理：

筛选所有距离待分析定日镜小于 r 的定日镜。

c.数学分析：

假设待分析定日镜其镜面单位法向量：

$\vec{n}_1=(n_{11}, n_{12}, n_{13})$ ，对待分析定日镜的长宽均进行等步长的分割。假设将长度均分为 m 段，宽度均分为 n 段，这在待分析定日镜上均匀产生了 $m * n$ 个交点，这些交点即为待分析的样本点。假设某一待分析的样本点坐标为 (x_0, y_0, z_0) ，假设某定日镜满足筛选条件，即可能对待分析定日镜造成阴影遮挡影响，其中心坐标为 (x_1, y_1, z_1) ，其镜面法向量 $\vec{n}_2=(n_{21}, n_{22}, n_{23})$ 。假设太阳光的单位方向向量 $\vec{s}=(s_x, s_y, s_z)$ 。由入射光方向的单位向量 $\vec{s}$ 和单位

法向量 $\vec{n}_1$ 可知，反射光线 $\vec{f}$ 满足：

$$\vec{f} = \vec{s} - 2(\vec{s} \cdot \vec{n}_1)\vec{n}_1$$

假设计算得到的反射光线 $\vec{f}=(f_x, f_y, f_z)$ 。由直线的参数方程可知：

$$\begin{array}{ll} \text{入射光线方程: } \begin{cases} x = x_0 + s_x t \\ y = y_0 + s_y t \\ z = z_0 + s_z t \end{cases} & \text{反射光线方程: } \begin{cases} x = x_0 + f_x t \\ y = y_0 + f_y t \\ z = z_0 + f_z t \end{cases} \end{array}$$

可能产生阴影遮挡的平面方程为：

$$n_{21}(x - x_1) + n_{22}(y - y_1) + n_{23}(z - z_1) = 0$$

联立线面方程，可计算出入射光线和反射光线与可能产生遮挡的平面的交点：

$$(x_2, y_2, z_2), (x_3, y_3, z_3)$$

将可能产生遮挡的平面投影至地平面上，获得点的投影坐标后，判断点是否在投影矩形内可由 Matlab 提供的 inpolygon 函数实现。

如果某个交点的投影在矩形内部，说明该待分析样本点的入射光线或反射光线被遮挡，该样本点无效。

d.遍历统计：

遍历该待分析定日镜上的所有样本点，统计无效的样本点数 k ，该定日镜的阴影遮挡效率即可近似为

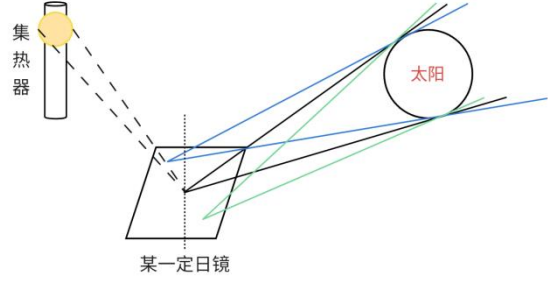
$$\eta_{sb} = 1 - \frac{k}{mn} \quad (13)$$



蒙特卡洛流程图

4. 截断效率（即溢出损失）的计算

由于太阳的入射光线是一束锥形光线，所以我们需要计算其半角展宽进而判断其影响。据资料，太阳的半径大约是 696300km，而日地距离大约是 149579870km，计算得太阳的半角展宽 α_h 约为 0.00465504rad，估算后为 4.65mrad。而这个大小的半角展宽会使得在定日镜中心距离距吸收塔高度为 360m 时产生 $r = d \tan \alpha_h \approx 1.67\text{m}$ 的光圈。这个半径大小的光圈在实际已经不能忽略。因此，在本题的计算过程中，我们将太阳光视为半角展宽为 4.65mrad 的光锥。



由于太阳光在反射时会产生发散现象，加之定日镜自身因距离过远或者因为镜面的个体差异而产生的追踪或精度的问题，导致一些光束纵使不受阴影遮挡的约束，也无法被集热器吸收，从而造成能量的损失。由此引出截断效率的概念，截断效率是指集热器接收的能量占定日镜场反射出的太阳光能量的百分比。计算公式为：

$$\eta_{\text{trunc}} = \frac{\text{集热器接收能量}}{\text{镜面全反射能量} - \text{阴影遮挡损失能量}}$$

为了计算定日镜场的截断效率，理论模型的建立，我们通常采用光线追迹法，追迹一束太阳光中任意一条光线在经过定日镜反射后，落入集热器的位置。从而得到定日镜的截断效率和集热器上的能流密度分布。

从地球上观察到的太阳的像，是一个圆形，其上的能量分布是不均匀的，由内而外逐渐减小。针对光斑能量的建模有多种不同形式，这里选用较为常用的一个经验公式，以夹角的大小来确定位置，进而来描述光斑上的能流密度：

$$S(\alpha) = \begin{cases} S_0[1 - \lambda(\alpha/\alpha_h)^4], & \alpha \leq \alpha_h \\ 0, & \alpha > \alpha_h \end{cases} \quad (14)$$

其中， $S(a)$ 是指太阳圆盘上某一点的能流密度，单位为 W/m^2 ；常量 $\lambda = 0.5138$ ；太阳光锥的半角展宽 $\alpha_h = 0.465\text{mrad}$ ； α 是光斑上一点到观察点的连线同光斑中心点到观察点连线的夹角，当 α 大于 α_h 时， $S(a)$ 等于 0，即认为光斑外部无能量； S_0 为光斑中心点的能量，其单位为 W/m^2 ，计算公式：

$$S_0 = \frac{G_{\text{bn}}}{\pi * R_s * (1 - \frac{\lambda}{3})} \quad (15)$$

其中， R_s 为光斑的半径，可以通过半角展宽和观察点到光斑中心的距离计算出， G_{bn} 则是反射光锥的能量，计算公式：

$$G_{\text{bn}} = \frac{DNI * S_h * \eta_{\cos} * \rho}{N_{\text{lot}}} \quad (16)$$

其中，DNI 是法向直接辐射辐照度，单位是 kW/m^2 ，计算公式为

$$\begin{aligned} DNI &= G_0 * [a + b \exp(-\frac{c}{\sin \alpha_s})], \\ a &= 0.4237 - 0.00821(6 - H)^2, \\ b &= 0.5055 + 0.00595(6.5 - H)^2, \\ c &= 0.2711 + 0.01858(2.5 - H)^2, \end{aligned} \quad (17)$$

G_0 为太阳常数，其值取为 $1.366 \text{ kW}/\text{m}^2$ ， H 为海拔高度（单位：km）。 S_h 为每面定日

镜的镜面面积， $\eta_{\cos}$ 是余弦效率， ρ 是镜面反射率，本题为

0.92， N_{lot} 是在每面定日镜上均匀投撒的大量随机点。

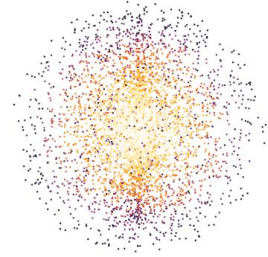
我们可以再设 N_{tr} 是每个反射光锥中，我们追踪的光线数，

整理公式（14）、（15）、（16）、（17）可以求得光斑上单个面积微元的能量数值的解析解，也即反射光锥中被追迹的单根光线能量 dE ，图 n 即为用解析解模拟出的光斑能流密度，数据点越密集，色调越偏向暖色调，能流密度越大，具体解析公式如下：

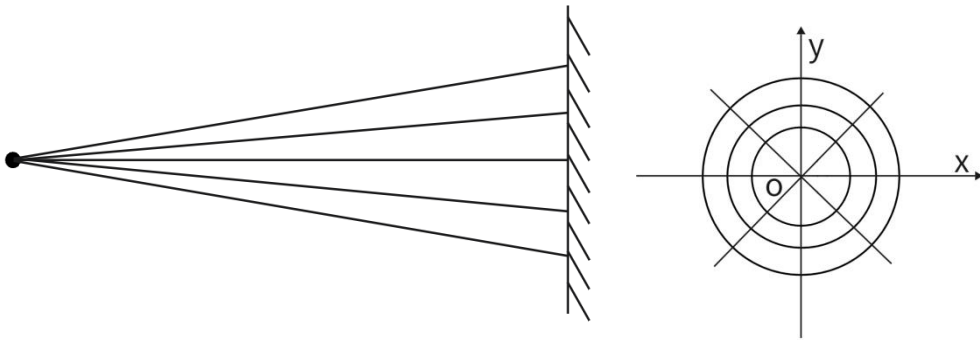
$$dE = S_0 [1 - \lambda(\frac{\alpha}{\alpha_h})^4] ds = \frac{DNI \cdot S_h \cdot \eta_{\cos} \cdot \rho}{N_{\text{lot}} \cdot N_{\text{tr}} \cdot (1 - \frac{\lambda}{3})} [1 - \lambda(\frac{\alpha}{\alpha_h})^4] \quad (18) \quad [3]$$

由表达式可知， dE 的解析解中包含角度因素的考虑，也就是说，对于一束光锥在半角展宽方向的均匀划分，其各处能量是不相等的。

正因如此，截断损失的能量涉及到复杂的积分运算。但在此可对截断效率的模型进行相应简化：假设其各处能量均匀分布。于是在太阳发散角内均匀追迹若干光线作为入射光线，均匀追迹的方式是在半角展宽方向以均匀角度进行步长划分，在周向方向上，也同样以均匀角度进行步长划分。即用光线的数量替代光线的能量，对问题的计算进行简化。^[2]



光斑能流密度



在此基础上，对于某一束符合要求的光锥，假设其轴线的单位方向向量为 $\vec{s} = (s_x, s_y, s_z)$ 。对于半角展宽方向，假设其在半角展宽方向前进了 $\Delta\theta$ ，则求解方程组

$$s_x(x - s_x) + s_y(y - s_y) + s_z(z - s_z) = 0$$

$$(x - s_x)^2 + (y - s_y)^2 + (z - s_z)^2 = \tan^2 \Delta\theta$$

得到一个符合约束的解向量 $\vec{u} = (x, y, z)$ 。通过罗德里格旋转公式得该到解向量沿轴线 $\vec{s}$ 旋转

角度 $\Delta\theta$ 后的向量为

$$\vec{u}' = \vec{u} \cos \Delta\theta + (\vec{s} \cdot \vec{u}) \vec{s} (1 - \cos \Delta\theta) + (\vec{s} \times \vec{u}) \sin \Delta\theta$$

$\Delta\theta$ 在 $[0, 2\pi]$ 上均匀取值, 即可实现光束的周向遍历。 $\Delta\theta$ 在半角展宽上均匀取值, 即可实现光束在半角展宽方向上的遍历。

通过上述方法, 可以得到沿半角展宽方向和周向方向上划分的全部向量的坐标表示。

得到的向量集合即为追迹的入射光线的方向向量。已知所有追迹的光线的方向向量 $\vec{s}$ 及镜面的法向量 $\vec{n}$, 由公式

$$\vec{f} = \vec{s} - 2(\vec{s} \cdot \vec{n})\vec{n}$$

可得反射光线的向量表示。

假设计算得到的反射光线 $\vec{f} = (f_x, f_y, f_z)$, 入射点坐标为 (x_0, y_0, z_0) , 则有反射光线方程:

$$\begin{aligned} x &= x_0 + f_x t \\ y &= y_0 + f_y t \\ z &= z_0 + f_z t \end{aligned} \quad (19)$$

由题知在镜面坐标系下, 集热器表面的方程为:

$$\begin{aligned} x^2 + y^2 &= \frac{49^2}{4} \\ z &\in [76, 84] \end{aligned} \quad (20)$$

通过联立线和面的方程, `vpasolve` 函数可求得 x, y, z 的数值解。若求得的解满足集热器表面的方程, 则表明该光线被集热器成功吸收。假设追踪的光线数量为 m , 被成功吸收的光线数量为 n , 则有

$$\eta_{\text{trunc}} = \frac{m}{n} \quad (21)$$

5.1.2 模型的求解

Step1: 光学效率的计算:

想要计算光学效率, 需要先计算出阴影遮挡效率、余弦效率、大气透射率、集热器截断效率以及镜面反射率。

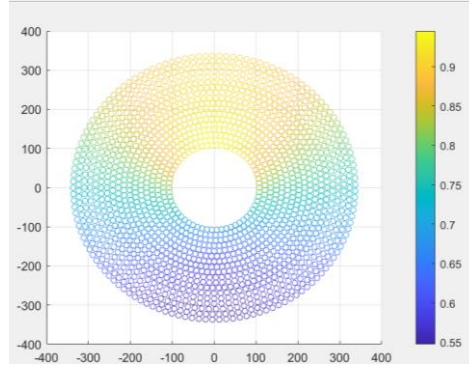
a. 镜面反射率 η_{ref} : 镜面反射率 η_{ref} 为常数 0.92;

b. 余弦效率 η_{cos} : 由公式 (5)、(6) 获得太阳入射光的方向向量和每一面定日镜各自的法向量, 再将其带入公式 (10) 当中:

$$\eta_{\text{cos}} = \frac{|\vec{n}| |\vec{s}| - |\vec{n} \cdot \vec{s}|}{|\vec{n}| |\vec{s}|}$$

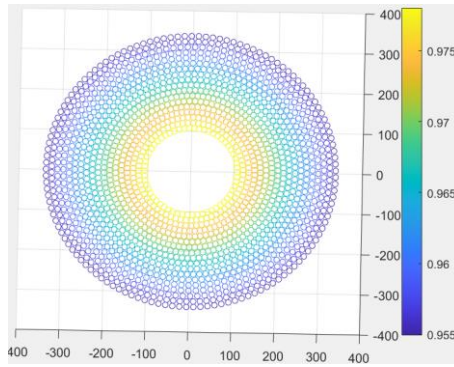
再通过公式 (5)、(6) 将所有“年均”指标的计算时点带入式中, 即可得到对于任意定日镜在这一时刻的余弦效率, 遍历所有定日镜即可得到它们各自的余弦效率。经计算, 第一

一间中定日镜场的年均余弦效率为 $\eta_{\text{cos}} = 0.7558$



定日镜场内各点的年均余弦效率

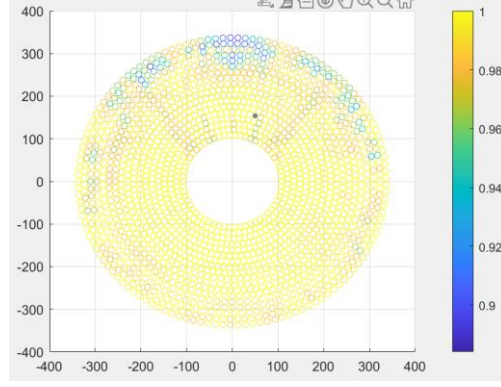
c.大气透射率 η_{at} ：由公式（9）可知定日镜中心在镜面坐标系中的坐标，由于集热器中心内在 z 轴上，可表示为 $(0, 0, 80)$ ，通过公式（12），计算出各个定日镜的中心到集热器中心的距离，再通过公式（11），就直接计算出了大气透射率 η_{at} 。经计算，第一问中定日镜场的年均大气透射率为 $\eta_{at} = 0.9652$



定日镜场内各点年均大气透射率

d.阴影遮挡效率 η_{sb} ：我们对每一面定日镜的镜面采用蒙特卡洛算法，均匀生成散点，并对入射与反射的光线进行计算。同时，将可能产生遮挡的平面及光线与平面的交点投影到地平面上，运用Pnpoly算法判断生成的光线是否被遮挡。最后通过公式（13），经过计算机提供的模拟数据得到阴影遮挡效率 η_{sb} 。经计算，第一问中定日镜场的年均阴影遮挡率为

$$\eta_{sb} = 0.9916$$



定日镜场内各点年均阴影遮挡率

e. 截断效率 η_{trunc} ：利用光线追踪法，将每一面定日镜上反射的每一束锥形光束进行简化，视作若干光线，然后对生成的光线进行追踪，通过公式（19）、（20）判断光线是否打在了集热器上，以光线的数量代替能量的大小，通过公式（21），经过计算机提供的模拟数据得到阴影遮挡效率经计算，第一问中定日镜场的年均截断效率为 $\eta_{\text{trunc}} = 0.9674$ 。

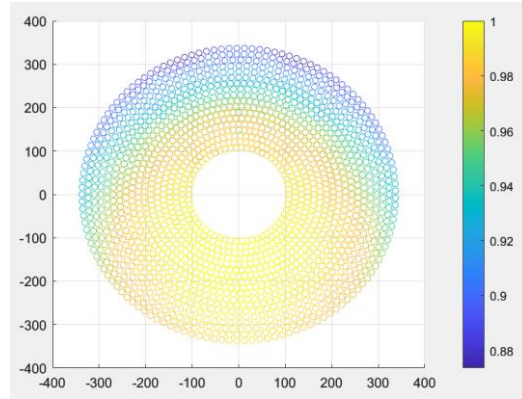


图 8 定日镜场内各点的年均截断效率

Step2:输出热功率的计算:

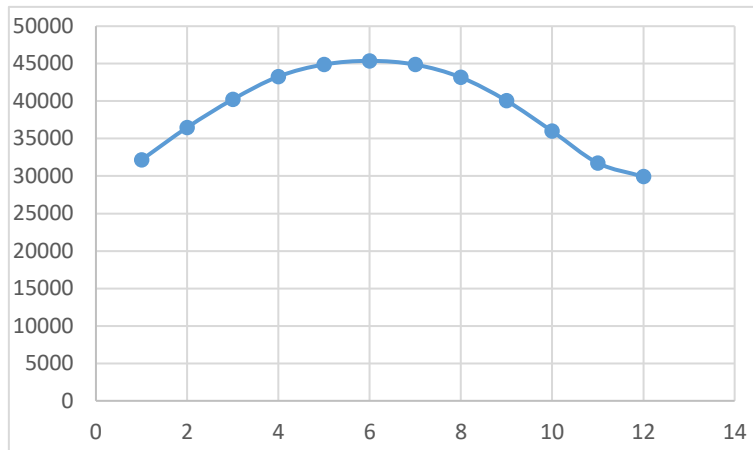
已知镜场坐标海拔均为 3000m，由公式（17）可知

$$\text{DNI} = 1.366 * [0.34981 + 0.5783875 \exp(-\frac{0.275745}{\sin \alpha_s})]$$

通过公式（5）将所有“年均”指标的计算时点带入式中，即可得到对于任意一面定日镜在

这一时刻的 DNI，遍历所有定日镜即可。最后通过公式 $E_{\text{field}} = \text{DNI} * \sum_i^N A_i * \eta_i$ 计算出

E_{field} ，第一问中定日镜场的年平均输出热功率为 $E_{\text{field}} = 39.0112\text{kW}$ 。



定日镜场总功率随时间的变化

5.1.3 求解结果的分析

由图进行分析：定日镜中心需要将入射的太阳光反射向集热器的中心，所以定日镜彼此之间的余弦差异就只和其所处与太阳的相对位置相关。通过对余弦效率差异的统计图的直观观察得，程序结果和直观理论是大体符合的。

由图可以看出其基本呈现越靠近冬至日（12月22日），数值越小，越靠近夏至日（6月22日），数值越大的规律。在第一问的设定下，影响光学效率的最主要的因素是余弦效率，影响输出热功率的最主要因素是DNI，而影响这两者大小的因素又都是太阳高度角，所以月平均数据会呈现类似抛物线的形状。程序结果和理论分析大体相符合。

5.2 问题二模型的建立与求解

5.2.1 模型的建立

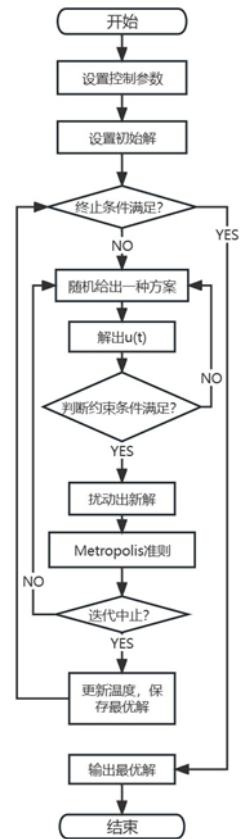
模拟退火算法是一种随机寻优算法，其出发点是基于物理中固体物质的退火过程与一般组合优化问题之间的相似性。模拟退火算法从某一较高初温出发，伴随温度参数的不断下降，结合概率突跳特性在解空间中随机寻找目标函数的全局最优解，即在局部最优解能概率性地跳出并最终趋于全局最优。考虑到模拟退火算法在处理多变量优化问题时的近似解的精度较高及寻求较优近似解所需花费的时间相较一些复杂优化算法而言较少，我们决定针对题目二采用模拟退火法进行最优化布局的寻找。

题目二包含吸收塔坐标、定日镜尺寸、安装高度、定日镜数目、定日镜位置五个决策变量。如果使用最优化算法同时将五个决策变量纳入随机扰动，所需的时间无疑是巨大且得到的解在有限的时间内是不稳定的。因此我们考虑先缩小参数的变化范围。

吸收塔的坐标是最先需要缩小范围的参数，因为吸收塔坐标的改变对于解的变化是影响较大的。由于定日镜场为对称的图形，因此我们考虑先在一个定日镜分布较均匀的定日镜场内，通过遍历吸收塔的可能位置，计算在每个位置上定日镜场对应的年均输出功率，从而确定吸收塔的初始坐标点使得对应年均输出功率最大。

在吸收塔初始坐标确定之后，通过改进 Campo 布置方法布置定日镜。Campo 是由 FJ Collado 等人提出的一种圆形定日镜场的布置方式，通过该方法可以生成灵活且规则的径向交错的密集型定日镜场。Campo 规则布置镜场首先从第一个布置区域的第一行开始。第一行的半径 R_1 由第一行的定日镜数量 N_{hel1} 计算得出。布置过程开始时，首环第一个定日镜放置在镜场的正北方向，其余定日镜以不发生机械碰撞为原则进行周向均匀布置。第二行定日镜数目与第一行相同，与第一行定日镜交错放置且使相邻行定日镜的特征圆相切。通过软件仿真可知，这种密集型镜场的阴影遮挡效率很低但其余光学效率却很高。但在模拟退火法中，可以通过对定日镜尺寸的扰动，增大阴影遮挡效率，从而达到阴影遮挡和其余光学效率的平衡，使得所寻求的解较优。

确定吸收塔的初始坐标和定日镜的布置后，通过设置初始温度，对定日镜的尺寸、安装高度、吸收塔的坐标进行微扰，并计算扰动后的年均输出热功率和单位面积年均输出热功率，并以对应概率接受新解。经过退火的过程后获取在满足年均输出热功率的约束下单位面积年均输出热功率最高的方案对应的定日镜尺寸、安装高度、吸收塔坐标。



5.2.2 模型的求解

Step1:确定吸收塔初始坐标

由于不同目标地点的经纬度的区别，太阳光的整体照射方向在不同地点有着较大的不同，因此吸收塔的初始坐标会对定日镜场的总体热功率产生非常大的影响。考虑到整个定日镜场为正圆形，具有高度的对称性，因此，可以在固定定日镜的总体布局方式的基础上便利吸收塔的位置，观察吸收塔的坐标对于定日镜场总体热功率的影响。

由于随机生成定日镜布局的不确定性和且其他经典布局方式 Campo 布局方式存在过于紧密的问题，此处选择均匀扩散式定日镜布局方式，通过控制扩散速度和行间距，可以生成疏密程度适中，具有高度对称性的定日镜分布。

(1) 均匀扩散式定日镜布局的生成：

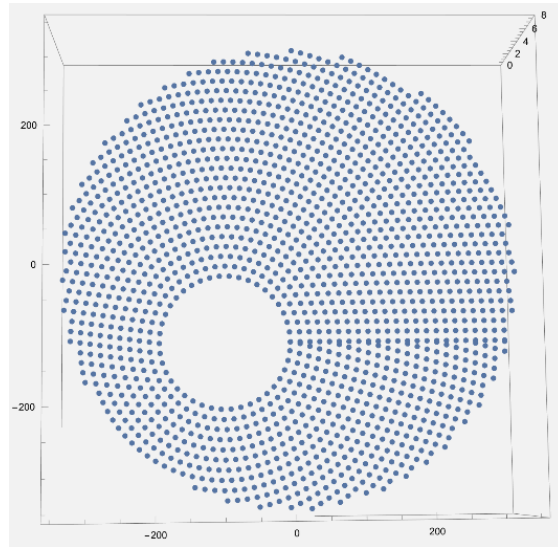
1. 在满足相邻定日镜距离需要大于定日镜宽度+5 的前提下，确定最内圈定日镜的数

目，在最内圈均匀生成定日镜。

2.根据行间距确定下一圈定日镜和吸收塔的距离，并根据扩散速度线性增加下一圈定日镜的数目，在下一圈均匀生成定日镜。

3.迭代直至定日镜和吸收塔的距离大于吸收塔和边界的最大距离。

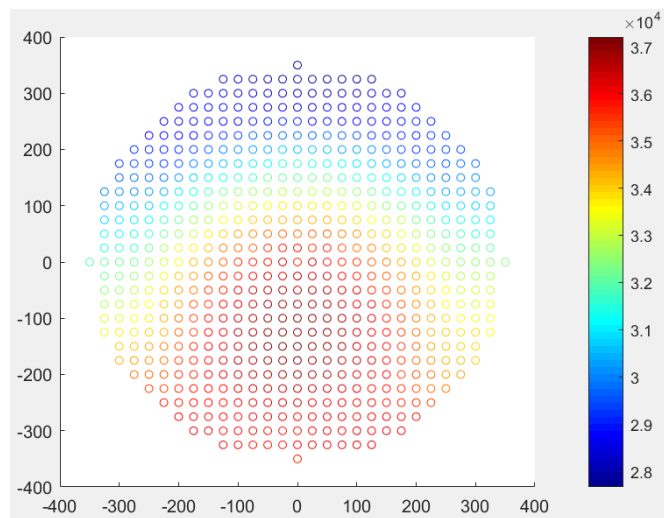
通过以上算法，可以基于任何吸收塔坐标生成以吸收塔为圆心，高度对称且相似的定日镜布局，如下图所示：



以(-100,-100)为吸收塔坐标生成的定日镜布局

(2) 吸收塔的初始坐标遍历：

以东为 x 轴正方向和北为 y 轴正方向等间隔将定日镜场分割为均匀网格，遍历吸收塔坐标在每个网格中心的情况，生成对应定日镜布局并计算年平均热功率，将输出结果可视化作出如下三维图：



定日镜在不同位置下的年平均热功率

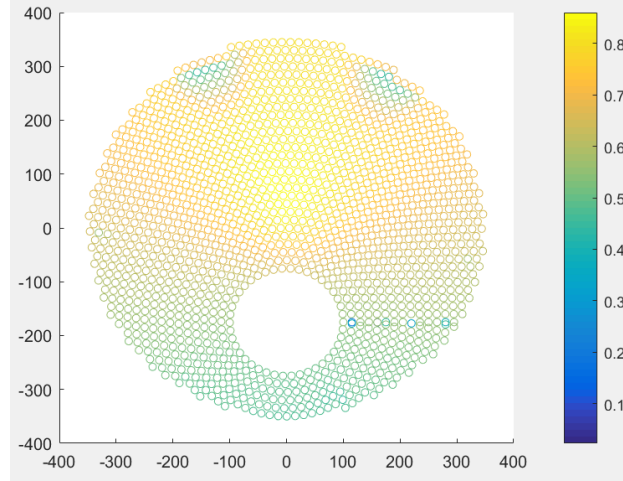
可以观察到，当吸收塔坐标位于(0, -175)时，定日镜场的年平均热功率达到最大，因此考虑将该点作为后续智能迭代优化时吸收塔的初始坐标。

Step2:优化算法的选择与实施

本问题有着庞大的决策变量和较高的目标函数计算时间，基于蚁群或粒子群的智能优化算法极易收敛至局部极值点，且收敛速度有着很大的限制，为了实现在短时间内收敛出全局较优解，考虑使用模拟退火算法与理论分析建模结合，在较优的初始参数值下进行扰动，从而在保证收敛速度的同时兼顾最优解的质量。

(1) 前置理论分析

以(0, -175)为吸收塔坐标，生成均匀扩散式定日镜布局并计算每个定日镜的光学效率，将输出结果可视化并作图：



定日镜光学效率分布图

观察年平均热功率分布图，图中黄色区域定日镜光学效率较高，因此可以在这类区域安排较密的定日镜分布，而图中蓝绿色区域定日镜年光学效率较低，因此可以在这类区域安排较为稀疏的定日镜分布，从而在确保额定功率满足的前提下尽可能减少定日镜的个数，从而实现定日镜单位镜面面积年平均输出热功率的最大化。

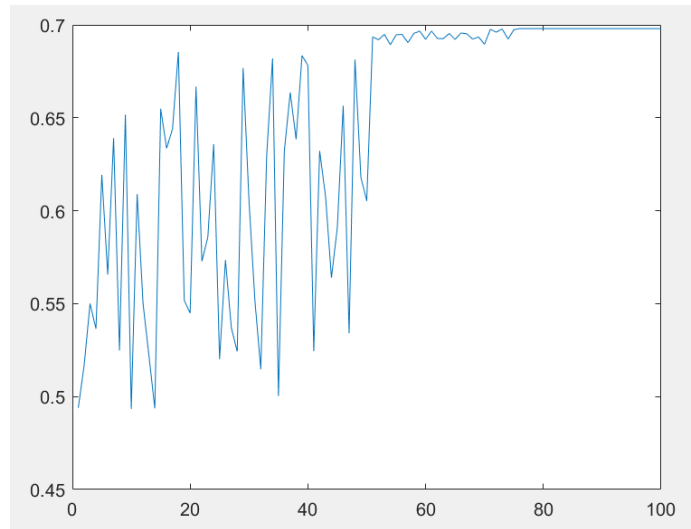
考虑上述因素，构造改进 Campo 布置方法行间距为 5 为首项，0.1 为公差的等差数列：

$$D_i = D_1 + d \cdot i, (d = 0.1, D_1 = 5)$$

这样的布局方法可以很好地增大光学效率高的区域的定日镜密度，减小光学效率低的区域的定日镜密度。

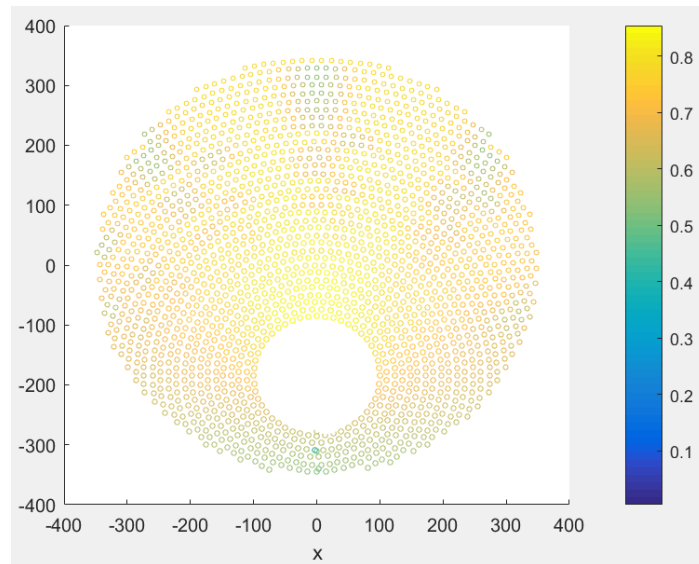
(2) 模拟退火算法求解

将长和宽 $W = 6$ ， $L = 6$ ，中心高度 $h = 4$ 作为初始参数传入，为了加快收敛速度，选定初始温度为 100，冷却系数为 0.95，每次基于正态分布，对各参数进行不超过当前值 10% 的扰动，迭代至收敛，其模拟退火收敛曲线如图所示：



模拟退火收敛曲线

最后得到较优解，其定日镜分布及其光学效率如图所示：



优化后的定日镜及其光学效率分布图

最终部分参数选值如下表：

吸收塔位置坐标	定日镜尺寸(宽, 高)	定日镜安装高度(m)	定日镜总面数	定日镜总面积(m^2)
(0, -188.19)	(7.703, 6.1742)	3.4744	1832	87129.92

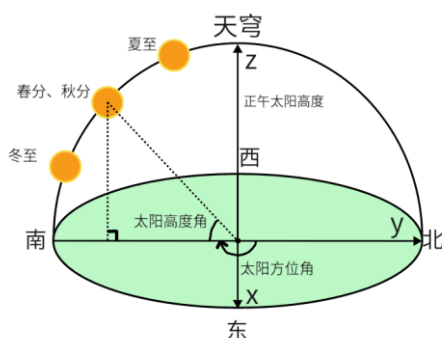
各结果值如下表：

年平均光学效率	年平均余弦效率	年平均阴影遮挡效率	年平均截断效率	年平均输出热功率(MW)	单位面积镜面年平均输出热功率(kW/m^2)
0.7197	0.8415	0.9646	0.96705	60.7191	0.6968

5.2.3 模型的检验

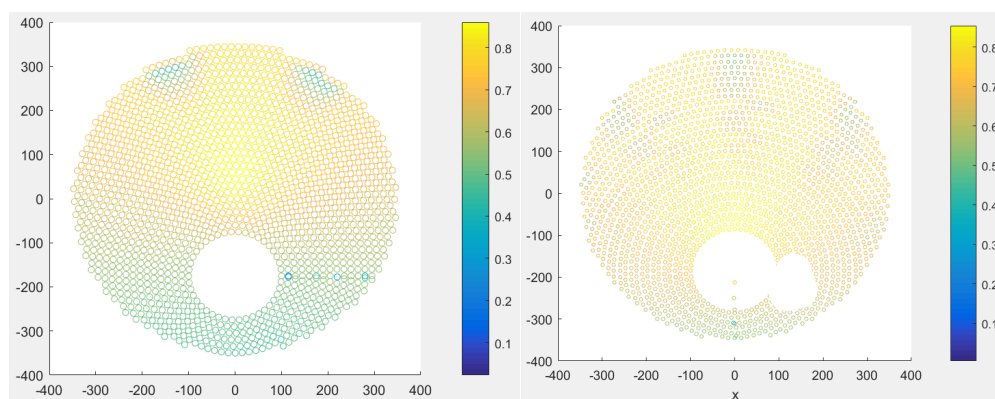
(1) 吸收塔位置坐标合理性分析:

分析考虑吸收塔位置坐标(0, -188.19)，我们可以发现其位于中心偏南处，导致的原因是太阳恒定东升西落，在东西方向上的光照对称，因此吸收塔并未在东西方向上发生偏移，同时由于目标地点选址位于东经 98.5 °，北纬 39.4 °，该地位于北回归线以北，日照方向在大多情况下偏南（如下图所示），因此偏南的吸收塔位置可以减小更多定日镜的受直射时间，从而增大定日镜场整体的余弦效率和阴影遮挡效率。



(2) 定日镜布局合理性分析:

对比观察优化前后定日镜场光学效率分布:



优化前光学效率分布

优化后光学效率分布

我们可以显然发现优化后的定日镜场的光学效率分布更加均匀，各个区域光学效率差异减小。在与吸收塔直线距离较短的地方定日镜余弦效率和阴影遮挡效率较高，定日镜分布密度大，能够很好地利用光照能量，而在与吸收塔直线距离较大的地方定日镜分布密度减小，布局行间距增大，一方面减少了因定日镜分布密集造成的阴影遮挡损失，另一方面控制了场地上定日镜的总数量，保证了整个定日镜场单位面积的热功率。

5.3 问题三模型的建立与求解

5.3.1 模型的建立与求解:

该题在第二问的基础上定日镜各自的高度、尺寸也被作为参数加入最优化模型当中，求解难度进一步增大，考虑使用随机扰动搜索和启发式算法为核心算法。

Method1:随机扰动法

我们首先采用了蒙特卡洛随机扰动法。在该方法中，先建立一个基于问题二的基础模型，该模型描述了定日镜场的布局和属性。然后引入了随机性，选择任意十面定日镜，并微小地扰动其尺寸和高度。接着，重新计算系统的年均输出热功率，选择性接受新解。这一过程被重复多次，以获得多个随机样本的数据。尽管蒙特卡洛随机扰动法在理论上是一种可行的方法，但在实际应用中面临着一些挑战。首先，由于每次扰动的变化很小，需要

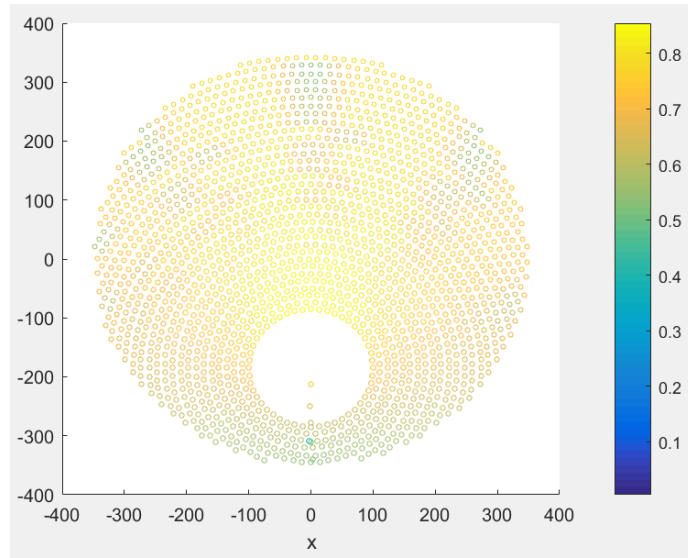
大量的计算资源来获得足够的样本数量，以确保结果的可靠性。这导致了计算效率的问题。其次，随机扰动可能导致不稳定的结果，使得优化过程变得复杂。

Method2:向阳山坡启发式算法

考虑到自然界中的向阳山坡，通常在阳光充足的地方植被茂盛。经由一观察启发，思考是否可以将山坡的地形特点应用于定日镜场系统，以提高能量利用效率。基本思路是在定日镜场系统中模拟山坡的地形，尤其是在阴影效率较低处。

观察第三题的结果，我们不难发现存在扇状的光学效率较低区域，考虑到在第三问中定日镜各自的高度、尺寸也作为参数加入最优化模型当中，因此可以基于向阳山坡的启发，构造高度由距离吸收塔较近处到距离吸收塔较远处为首项为 3.5，公差为 0.2 的等差数列：

$$H_i = H_1 + d \cdot i, (d = 0.2, H_1 = 3.5)$$



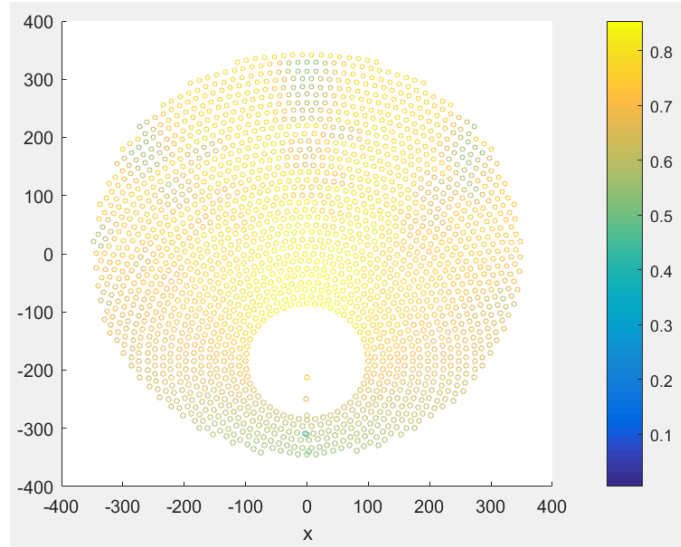
第三题结果

这样构造出的坡型结构可以很好地提升图中扇形区域的阴影遮挡效率，同时注意到在离吸收塔远的地方可以减小定日镜面积从而进一步减少阴影遮挡损失，获取更高的单位面积年平均热功率，构造定日镜面积由距离吸收塔较近处到距离吸收塔较远处为首项为 60，公差为-1 的等差数列：

$$S_i = S_1 + d \cdot i, (d = -1, S_1 = 60)$$

之后运用模拟退火法，将系统等差数列中初始值与公差和其他第二问涉及参数作为本题参数加入最优化模型，迭代求解，以正态分布扰动各个参数，迭代求解最优值。

光学效率分布如图：



求得相关参数取值如下：

吸收塔位置 坐标	定日镜总面 数	定日镜总面 积 (m ²)
(0, -185.21)	1945	86556.75

求得相关值如下：

年平均光学 效率	年平均余弦 效率	年平均阴影 遮挡效率	年平均截断 效率	年平均输出 热功率 (MW)	单位面积镜 面年平均输 出热功率 (kW/m ²)
0.7239	0.8418	0.9698	0.967425	60.5728	0.6998

六、模型分析

6.1 由于太阳年而产生的误差的分析以及对模型灵敏度的检验

对于模型太阳高度角、太阳方位角的检验：

由于一个太阳年是 365.2422 天，而在公式（1）中，采用的是近似值 365，又因此影响到太阳高度角和太阳方位角的计算，进而也会影响单位面积的年平均输出热功率，我们现在需要分析的是，这 0.2422 天对太阳高度角的影响大小。以及对其各自添加 $\pm 1\%$ 的扰动，即 $0.04 \times \pi$ 的变化范围，直接观察单位面积的年平均输出热功率的变化程度。

Step1:

我们在网络上找一组数据作为理论依据

地点	日期	时间	经度	纬度	时区	高度角	方位角
北京某地	1990年1月1日	8:00	116.478°	39.8751°	+8	3.1427°	-56.1754°
南京某地	2012年3月4号	10:00	118.8°	32°	+8	40.016667°	-46.683333°

通过公式 (2)、(3)、(4) 我们可以分别计算出 365 与 365.2422 时相应角的度

	太阳方位角 (度)	太阳高度角 (度)	
365.2422	3.1427°	-56.1754°	北京
366.2422	3.1283°	-52.1888°	北京
365	40.016667°	-46.683333°	南京
366	40.014568°	-47.900821°	南京

数：

Step2:

进行分析发现，对于太阳高度角来说，会有前后 3° 左右的偏差，大概是 0.01；对于太阳方位角来说，因为小于 0.003，所以完全可以忽略不计。再由公式 (5) 可得，太阳高度角的偏差并不会产生致命的误差，对最终的计算结果也不会有太大的影响。因此得出结论模型具有一定的灵敏度，较为良好。

七、模型总结

7.1 模型的优点.

1. 题目二、三开放性较强。我们不拘泥于已有的算法，而是基于生活中的观察提出创新型的思考，诞生了一些有研究价值和实际效用的想法。
2. 本文采用的优化的模拟退火算法与改进的 campo 法，在众多诸如粒子群算法、自适应引力算法等算法中慎重选择，在有限的时间内实现了对全局最优解的更大概率的追求，而不是停留在局部最优解。
3. 本文使用大量图形进行论述，使得呈现结果直观、严密、有逻辑。

7.2 模型的缺点

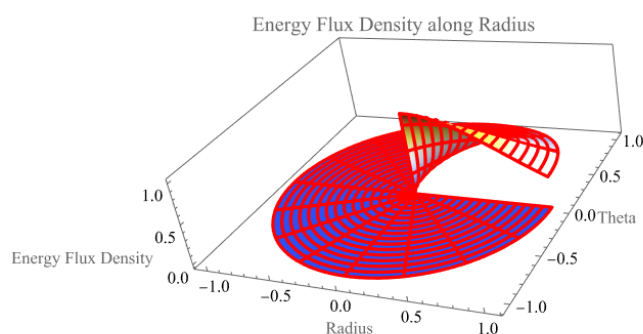
1. 在考虑模型计算时做出的近似和假设较多，可能最终会对精度造成一定的影响。

7.3 模型的思考与改进

7.3.1 模型对于集热器表面能流密度的忽略

题目为了降低难度，简化模型，进行了“控制系统根据太阳的位置实时控制定日镜的法向，使得太阳中心点发出的光线经定日镜中心反射后指向集热器中心”的设定，结合上文 5.1.1.2 中对光斑内能流密度的考虑可知，过集热器中心做与 xoy 平行的平面，与集热器的外表面相交形成的一圈圆环，是热量最为聚集之处，这会出现两点弊端，第一点，热量聚集之处升温快，温度极高，会影响集热器的零件寿命；第二点，相比集热器表面其他区域，中心的环形区域温度明显高出，这并不利于后续的热量传输，而且热传导过程也会消耗极多热量。

解决的方法有很多，最直接的一种是升级算法，让定日镜场的镜面中心反射的光线均匀分布在集热器表面，虽然截断效率会略微下降，但是对于能量转换来说，算法复杂的代价是能大幅提升集热器使用寿命和转换效率的。查阅资料，平面定日镜之外的曲面定日镜一样能够有效解决，曲面定日镜的金钱成本更大，但是却可以极好的汇聚光线，也就是说，本题中的几千面定日镜理论上是可以几十面大型的曲面定日镜替代的，截断效率也能因此提高。也可以考虑改变平面定日镜的型状，比如将矩形换成圆形，这样同样是第一种方法，截断效率的下降就可以大幅减少，考虑到制作成本，定日镜也可以改成六边形、八边形等，也可以达到接近的效果。



参考文献

- [1]. 刘建兴. 塔式光热电站光学效率建模仿真及定日镜场优化布置[D].兰州交通大学,2022.
- [2]. 张平等, 太阳能塔式光热镜场光学效率计算方法[J], 技术与市场, 2021, 28(6):5-8.
- [3]. 高维东. 塔式太阳能电站定日镜场调度优化研究[D].华北电力大学,2021.
- [4]. Carrizosa, Emilio et al. “An Optimization Approach to the Design of Multi-Size Heliostat fields.” (2014).

附录

支撑文件名称:

- 1.文件夹 problem1
- 2.文件夹 problem2
- 3.文件夹 problem3

Matlab
代码: % problem1 全部支撑程序 <pre> clc;clear; % 导入数据 global points; global shadow; points = % 导入数据 % 阴影遮挡矩阵 shadow = zeros(length(points), length(points)); shadow_id = ones(1, length(points)); for i = 1:length(points) for j = 1:length(points) if j == i continue; end if sqrt((points(j, 1)-points(i, 1))^2 + (points(j, 2)-points(i, 2))^2) < 20 shadow(i, shadow_id(i)) = j; shadow_id(i) = shadow_id(i) + 1; end end end % 时间矩阵 dd = [-59 -28 0 31 61 92 122 153 184 214 245 275]; tt = [9 10.5 12 13.5 15]; % 计算当前状态下所有法向量和所有矩形顶点 % 第一题长宽均为 6 ssum = 0; ssum_n = 0; for d_id = 1:12 </pre>


```

    for t_id = 1:5
        tmp = objectiveFunction1(dd(d_id), tt(t_id));
        ssum = ssum + tmp(1);
        ssum_n = ssum_n + mean(tmp(2));
    end
end

disp((ssum)/60);
disp((ssum_n/60));

%-----
%-----
%-----

%接受太阳高度角 alpha 和太阳方位角 gamma 返回每个定日镜的法向量
function result=NormalVector1(alpha,gamma)
    global points;
    Data = points;
    Inlight=[cos(alpha)*sin(gamma) cos(alpha)*cos(gamma) sin(alpha)];
    Inlight = Inlight/norm(Inlight);
    Target=[0 0 80];
    for i=1:length(points)
        Data(i,:)=Target(1,:) - Data(i,:);
        Data(i,:) = Data(i,:)/norm(Data(i,:));
    end
    result=zeros(length(points),3);
    for i=1:length(points)
        Data(i,:)=Data(i,:)+Inlight;
        result(i,:) = Data(i,:)/norm(Data(i,:));
    end
end

%-----
%-----
%-----

function res = objectiveFunction1(d, t)
    global points;
    global shadow;

    target = [0 0 80];

    sun_angle = sun_direction(d, t);
    sun_vec = sun_vector(d, t);
    rec = Coordinate1(6, 6, sun_angle(1), sun_angle(2));

```

```

norm_vec = NormalVector1(sun_angle(1), sun_angle(2));

% 记录余弦效率的矩阵
cos_val = zeros(1, length(points));

% 记录阴影遮挡效率的矩阵
shadow_val = zeros(1, length(points));

% 记录截断效率的矩阵
tunc_val = zeros(1, length(points));
% loss_val = zeros(1, length(points));

% 记录大气透射率的矩阵
air_val = zeros(1, length(points));

% 计算 DNI
temp_a = 0.4237-0.00821*(6-3)^2;
temp_b = 0.5055+0.00595*(6.5-3)^2;
temp_c = 0.2711+0.01858*(2.5-3)^2;
DNI = 1.366*(temp_a+temp_b*exp(-temp_c/sin(sun_angle(1))));

sum_cnt = 0;
sum_all_cnt = 0;
% 遍历每一个点，使用模拟光线的方法计算效率
for i = 1:length(points)
    cnt = 0;
    all_cnt = 0;
    tunc_cnt = 0;
    all_tunc_cnt = 0;

    min_x = min(rec(i, 1, :));
    max_x = max(rec(i, 1, :));
    d_x = (max_x-min_x) / 10;
    min_y = min(rec(i, 2, :));
    max_y = max(rec(i, 2, :));
    d_y = (max_y-min_y) / 10;
    min_z = min(rec(i, 3, :));
    max_z = max(rec(i, 3, :));
    d_z = (max_z-min_z) / 10;

    a = sun_vec;
    n = norm_vec(i, :);

```

```

% 计算余弦效率
cos_val(i) = abs(dot(a, n)/norm(a)/norm(n));

% 计算大气透射率
air_val(i) = 0.99321-0.0001176*norm([0,0,80] - points(i,:)) + 1.97*1e-8*norm(points(i,:) -
[0,0,80])^2;

mnt = 0;
for x = min_x:d_x:max_x
    for y = min_y:d_y:max_y
        % 判断是否在矩形上
        if inpolygon(x, y, [rec(i,1,1),rec(i,1,2),rec(i,1,3),rec(i,1,4)],
[rec(i,2,1),rec(i,2,2),rec(i,2,3),rec(i,2,4)]) == 0
            mnt = mnt+1;
            continue;
        end

% 计算当前网格下的 z 坐标
z = points(i, 3) - 1/n(3)*(n(1)*(x-points(i, 1)) + n(2)*(y-points(i, 2)));

all_cnt = all_cnt+1;
sum_all_cnt = sum_all_cnt+1;

% 计算反射光线向量
a_new = (a - dot(a, n/norm(n))*n/norm(n))- dot(a, n/norm(n))*n/norm(n);

% 标记是否产生了阴影损失
fflag = 0;
for j = 1:length(shadow(i, :))
    if shadow(i, j) == 0
        break;
    end

% 判定吸收塔阴影
if dot([target(1), target(2), points(i, 3)] - points(i, :), [sun_vec(1), sun_vec(2), 0]) <
0

% 生成并标准化目标向量
to_norm_vec = [sun_vec(1), sun_vec(2), 0];
to_norm_vec = to_norm_vec/norm(to_norm_vec);

% 计算与吸收塔平面的交点
[x0, y0, z0] = deal(target(1), target(2), target(3));
[n1, n2, n3] = deal(to_norm_vec(1), to_norm_vec(2), to_norm_vec(3));
lambda = -(n1*(x-x0) + n2*(y-y0) + n3*(z-z0)) / (sun_vec(1)*n1 +

```

```

sun_vec(2)*n2 + sun_vec(3)*n3);

    point_new = [x+lambda*sun_vec(1), y+lambda*sun_vec(2),
z+lambda*sun_vec(3)];

    % 判定是否被吸收塔遮挡
    if point_new(3) <= target(3)+4 && point_new(3) >= 0 && ...
        sqrt((point_new(1) - target(1))^2 + (point_new(2) - target(2))^2) < 3.5
        cnt = cnt+1;
        sum_cnt = sum_cnt+1;
        fflag = 1;
        break;
    end
end

% 判定阴影
tmp_j = shadow(i, j);
if dot(points(tmp_j, :) - points(i, :), [sun_vec(1), sun_vec(2), 0]) < 0
    [x0, y0, z0] = deal(points(tmp_j, 1), points(tmp_j, 2), points(tmp_j, 3));
    [n1, n2, n3] = deal(norm_vec(tmp_j, 1), norm_vec(tmp_j, 2),
norm_vec(tmp_j, 3));

    lambda = -(n1*(x-x0) + n2*(y-y0) + n3*(z-z0)) / (sun_vec(1)*n1 +
sun_vec(2)*n2 + sun_vec(3)*n3);
    point_new = [x+lambda*sun_vec(1), y+lambda*sun_vec(2),
z+lambda*sun_vec(3)];

    if inpolygon(point_new(1), point_new(2),
[rec(tmp_j,1,1),rec(tmp_j,1,2),rec(tmp_j,1,3),rec(tmp_j,1,4)],
[rec(tmp_j,2,1),rec(tmp_j,2,2),rec(tmp_j,2,3),rec(tmp_j,2,4)]) ~= 0
        cnt = cnt+1;
        sum_cnt = sum_cnt+1;
        fflag = 1;
        break;
    end
end

% 判定遮挡
[x0, y0, z0] = deal(points(tmp_j, 1), points(tmp_j, 2), points(tmp_j, 3));
[n1, n2, n3] = deal(norm_vec(tmp_j, 1), norm_vec(tmp_j, 2), norm_vec(tmp_j,
3));

    lambda = -(n1*(x-x0) + n2*(y-y0) + n3*(z-z0)) / (a_new(1)*n1 + a_new(2)*n2 +
a_new(3)*n3);
    point_new = [x+lambda*a_new(1), y+lambda*a_new(2), z+lambda*a_new(3)];
    if inpolygon(point_new(1), point_new(2),
[rec(tmp_j,1,1),rec(tmp_j,1,2),rec(tmp_j,1,3),rec(tmp_j,1,4)],
[rec(tmp_j,2,1),rec(tmp_j,2,2),rec(tmp_j,2,3),rec(tmp_j,2,4)]) ~= 0

```

```

        cnt = cnt+1;
        sum_cnt = sum_cnt+1;
        fflag = 1;
        break;
    end
end

% 判定截断损失
if fflag == 0
    tmp_loss = loss([x,y,z], sun_vec/norm(sun_vec), norm_vec(i,:));
    loss_val(i) = loss_val(i) + tmp_loss;
    all_tunc_cnt = all_tunc_cnt + 1;

    % 生成并标准化目标向量
    to_norm_vec = [(x-target(1)), (y-target(2)), 0];
    to_norm_vec = to_norm_vec/norm(to_norm_vec);

    % 计算与吸收塔平面的交点
    [x0, y0, z0] = deal(target(1), target(2), target(3));
    [n1, n2, n3] = deal(to_norm_vec(1), to_norm_vec(2), to_norm_vec(3));
    lambda = -(n1*(x-x0) + n2*(y-y0) + n3*(z-z0)) / (a_new(1)*n1 + a_new(2)*n2 +
a_new(3)*n3);

    point_new = [x+lambda*a_new(1), y+lambda*a_new(2), z+lambda*a_new(3)];

    % 判定是否能被集热器吸收
    ddistance = norm(target - [x,y,z]);
    rridus = ddistance/100*0.93/2;
    if point_new(3) <= target(3)-4 || point_new(3) >= target(3)+4 || ...
        sqrt((point_new(1) - target(1))^2 + (point_new(2) - target(2))^2) > 3.5
        tunc_cnt = tunc_cnt + 1;
    elseif norm(target - point_new) + rridus > 3.5
        tunc_cnt = tunc_cnt + 0.5*exp((norm(target - point_new) + rridus-
3.5)/rridus-1);
    end
end
end
end
if all_cnt == 0
    shadow_val(i) = 0;
else
    shadow_val(i) = 1 - cnt/all_cnt;
end
if all_tunc_cnt == 0
    tunc_val(i) = 0;

```

```

%         loss_val(i) = 0;
    else
        tunc_val(i) = 1 - tunc_cnt/all_tunc_cnt;
%         loss_val(i) = loss_val(i)/all_tunc_cnt;
    end
end

% 阴影效率
nsb = shadow_val;

% 镜面反射率
nref = 0.92*ones(1, length(points));

% 余弦效率
ncos = cos_val;

% 大气透射率
nair = air_val;

% 截断效率
ntrunc = tunc_val;

% 光学效率计算
nn = nsb.*nref.*ncos.*nair.*ntrunc;

% 输出热功率计算
Efield = DNI*sum(6*6*nn);

aver_Efield = Efield/length(points);

res = [Efield, mean(nn), mean(ncos), mean(nsb), mean(ntrunc)];

% % 创建平面高度场
% [xx, yy] = deal(points(:,1), points(:,2));
% zz = shadow_val';
% % 使用 surf 函数绘制三维图，并用颜色表示高度
% scatter3(xx, yy, zz, 10, zz, 'filled');colorbar; % 添加颜色条
%
% % 设置图的属性
% xlabel('X 轴');
% ylabel('Y 轴');
% zlabel('Z 轴');
% colorbar; % 添加颜色条

```



```

end

%-----
%-----
%-----

% 函数说明:
% 传入参数: 第一个传入参数是当前距离春分的天数, 例如 4 月 1 日距离春分 3 月 21 日为 11 天
%           第二个传入参数是当地时间, 以小数形式传入, 例如 9:30 即为 9.5
% 返回值: 返回值是一个 2*1 矩阵, 第一个值为太阳高度角, 第二个值为太阳方位角(注意太阳方位角并未考虑
正负之分, 实际上并不会对结果造成影响)
%

function res = sun_direction(dd, t)
    % 传入经度 ss 和纬度 rr, 当前距离春分的天数 D, 当前时间 tt
    ss = 116.478*pi/180;
    rr = 39.8751*pi/180;
    D = dd;
    tt = t;

    % 计算太阳赤纬角
    thgma = asin(sin(2*pi*D/365)*sin(2*pi/360*23.45));

    % 计算太阳时角
    w = pi/12*(tt-12);

    % 计算太阳高度角
    alpha_s = asin(sin(rr)*sin(thgma) + cos(thgma)*cos(rr)*cos(w));

    if w <= 0
        gamma_s = acos((sin(thgma)-sin(alpha_s)*sin(rr)) / (cos(alpha_s)*cos(rr)));
    else
        gamma_s = 2*pi - acos((sin(thgma)-sin(alpha_s)*sin(rr)) / (cos(alpha_s)*cos(rr)));
    end
    gamma_s = abs(gamma_s);
    res = [alpha_s, gamma_s];
end

%-----
%-----
%-----

```

```

% 函数说明:
% 传入参数: 第一个传入参数是当前距离春分的天数, 例如 4 月 1 日距离春分 3 月 21 日为 11 天
%           第二个传入参数是当地时间, 以小数形式传入, 例如 9:30 即为 9.5
% 返回值: 返回值是一个 2*1 矩阵, 第一个值为太阳高度角, 第二个值为太阳方位角(注意太阳方位角并未考虑
正负之分, 实际上并不会对结果造成影响)
%

function res = sun_vector(dd, t)
    % 传入经度 ss 和纬度 rr, 当前距离春分的天数 D, 当前时间 tt
    ss = 116.478*pi/180;
    rr = 39.8751*pi/180;
    D = dd;
    tt = t;

    % 计算太阳赤纬角
    thgma = asin(sin(2*pi*D/365)*sin(2*pi/360*23.45));

    % 计算太阳时角
    w = pi/12*(tt-12);

    % 计算太阳高度角
    alpha = asin(sin(rr)*sin(thgma) + cos(thgma)*cos(rr)*cos(w));

    % 计算太阳方位角
    if w < 0
        gamma = acos((sin(thgma)-sin(alpha)*sin(rr)) / (cos(alpha)*cos(rr)));
    else
        gamma = 2*pi - acos((sin(thgma)-sin(alpha)*sin(rr)) / (cos(alpha)*cos(rr)));
    end
    gamma = abs(gamma);
    res = [-cos(alpha)*sin(gamma) cos(alpha)*cos(gamma) sin(alpha)];
end

%-----
%-----
%-----

% problem2 主要支撑程序
clc;clear;

data = [];

```

```

for i = -350:25:350
    for j = -350:25:350
        if i^2 + j^2 > 350^2
            continue;
        end
        current_solution = [i, j];
        clear xx;
        xx = [];
        ii = 1;
        for r = 100:15:750
            for theta = 0:2*pi/(2*pi*r/15):2*pi
                if (current_solution(1) + r*cos(theta))^2 + (current_solution(2) + r*sin(theta))^2 <
350^2
                    xx = [xx; [current_solution(1) + r*cos(theta), current_solution(2) + r*sin(theta),
4]];
                    ii = ii+1;
                end
            end
        end
        current_cost = objectiveFunction2(current_solution(1), current_solution(2), 6, 6, xx);
        current_cost = current_cost(1); % 计算年平均热功率
        data = [data; [i,j,current_cost]];
    end
end

disp(data);

%-----
%-----
%-----

clc;clear;

% 初始解
current_solution = [0, -185, 8, 7.5, 3.75, linspace(5,16.5,150)];
current_point_set = generate_new([current_solution(1), current_solution(2)], current_solution(3),
current_solution(4), current_solution(5), current_solution(6:end));
current_cost = objectiveFunction2(current_solution(1), current_solution(2), current_solution(3),
current_solution(4), current_point_set); % 计算初始解的成本
current_cost = -current_cost(2);

% 初始温度和降温率
initial_temperature = 100;

```

```

cooling_rate = 0.95;

% 定义最大迭代次数
max_iterations = 100;

% 初始化最佳解和最佳成本
best_solution = current_solution;
best_cost = -current_cost;

% 数据存储
data = zeros(1, max_iterations);
data_id = 1;

% 开始模拟退火主循环
for iteration = 1:max_iterations
    % 生成一个新的解
    new_solution = current_solution + [0, randn(), 0.05*randn(), 0.05*randn(), 0.05*randn(),
linspace(1,100,150)*0.005*randn()];
    new_point_set = generate_new([new_solution(1), new_solution(2)], new_solution(3),
new_solution(4), new_solution(5), new_solution(6:end));
    new_cost = objectiveFunction2(new_solution(1), new_solution(2), new_solution(3),
new_solution(4), new_point_set);
    if new_cost(1) > 60000
        new_cost = -new_cost(2);
    else
        new_cost = 0;
    end

    % 计算成本变化
    cost_change = new_cost - current_cost;

    % 如果新解更好或者以一定概率接受更差的解
    if cost_change < 0 || (rand() < exp(-cost_change / initial_temperature))
        current_solution = new_solution;
        current_cost = new_cost;

        % 更新最佳解
        if current_cost < best_cost
            best_solution = current_solution;
            best_cost = current_cost;
        end
    end

    % 降温

```

```

        initial_temperature = initial_temperature * cooling_rate;

        % 数据记录
        data(data_id) = current_cost;
        data_id = data_id+1;
    end

    % 输出结果
    fprintf('最佳解: %.4f %.4f\n', best_solution);
    fprintf('最佳成本: %f\n', best_cost);

    %-----
    %-----
    %-----

    % 传入参数分别是中心塔坐标，定日镜长宽和定日镜中心坐标集合
    function res = objectiveFunction2(cen_x, cen_y, L, K, xx)
        target = [cen_x cen_y 80];
        points = xx;

        % 阴影遮挡矩阵
        shadow = zeros(length(points), length(points));
        shadow_id = ones(1, length(points));
        for i = 1:length(points)
            for j = 1:length(points)
                if j == i
                    continue;
                end
                if sqrt((points(j, 1)-points(i, 1))^2 + (points(j, 2)-points(i, 2))^2) < L+10
                    shadow(i, shadow_id(i)) = j;
                    shadow_id(i) = shadow_id(i) + 1;
                end
            end
        end

        % 时间矩阵
        dd = [0 92 184 275];
        tt = [9 10.5 12 13.5 15];

        ssum = 0;
        ssum_n = 0;
        all_nn = 0;

```

```

all_ncos = 0;
for d_id = 1:4
    for t_id = 1:5
        sun_angle = sun_direction(dd(d_id), tt(t_id));
        sun_vec = sun_vector(dd(d_id), tt(t_id));
        % rec = Coordinate2(points, L, K, sun_angle(1), sun_angle(2));
        norm_vec = NormalVector2(points, target, sun_angle(1), sun_angle(2));

        % 记录余弦效率的矩阵
        cos_val = zeros(1, length(points));

        % 记录阴影遮挡效率的矩阵
        shadow_val = zeros(1, length(points));

        % 记录截断效率的矩阵
        tunc_val = ones(1, length(points));

        % 记录大气透射率的矩阵
        air_val = zeros(1, length(points));

        % 计算 DNI
        temp_a = 0.4237-0.00821*(6-3)^2;
        temp_b = 0.5055+0.00595*(6.5-3)^2;
        temp_c = 0.2711+0.01858*(2.5-3)^2;
        DNI = 1.366*(temp_a+temp_b*exp(-temp_c/sin(sun_angle(1))));

        % 遍历每一个点，使用平面投影法计算阴影效率
        for i = 1:length(points)
            a = sun_vec;
            n = norm_vec(i, :);

            % 计算余弦效率
            cos_val(i) = abs(dot(a, n)/norm(a)/norm(n));

            % 计算大气透射率
            air_val(i) = 0.99321-0.0001176*norm([0,0,80] - points(i,:)) + 1.97*1e-
8*norm(points(i,:) - [0,0,80])^2;

            [x,y,z] = deal(points(i,1), points(i,2), points(i,3));

            % 计算反射光线向量
            a_new = (a - dot(a, n/norm(n))*n/norm(n))- dot(a, n/norm(n))*n/norm(n);

```

```

        for j = 1:length(shadow(i, :))
            if shadow(i, j) == 0
                break;
            end
            % 判定阴影
            tmp_j = shadow(i, j);
            if dot(points(tmp_j, :) - points(i, :), [sun_vec(1), sun_vec(2), 0]) < 0
                [x0, y0, z0] = deal(points(tmp_j, 1), points(tmp_j, 2), points(tmp_j, 3));
                [n1, n2, n3] = deal(norm_vec(tmp_j, 1), norm_vec(tmp_j, 2),
norm_vec(tmp_j, 3));
                lambda = -(n1*(x-x0) + n2*(y-y0) + n3*(z-z0)) / (sun_vec(1)*n1 +
sun_vec(2)*n2 + sun_vec(3)*n3);
                point_new = [x+lambda*sun_vec(1), y+lambda*sun_vec(2),
z+lambda*sun_vec(3)];

                dis = norm(point_new-[x0,y0,z0]);
                hig = abs(point_new(3)-z0) * 1/(n1^2+n2^2);
                reHig = K-hig;
                reWid = L-sqrt(dis^2-hig^2);
                if reHig > 0 && reWid > 0
                    shadow_val(i) = shadow_val(i) + reHig*reWid/(L*K);
                end
            end

            % 判定遮挡
            [x0, y0, z0] = deal(points(tmp_j, 1), points(tmp_j, 2), points(tmp_j, 3));
            [n1, n2, n3] = deal(norm_vec(tmp_j, 1), norm_vec(tmp_j, 2), norm_vec(tmp_j,
3));
            lambda = -(n1*(x-x0) + n2*(y-y0) + n3*(z-z0)) / (a_new(1)*n1 + a_new(2)*n2 +
a_new(3)*n3);
            point_new = [x+lambda*a_new(1), y+lambda*a_new(2), z+lambda*a_new(3)];

            dis = norm(point_new-[x0,y0,z0]);
            hig = abs(point_new(3)-z0) * 1/(n1^2+n2^2);
            reHig = K-hig;
            reWid = L-sqrt(dis^2-hig^2);
            if reHig > 0 && reWid > 0
                shadow_val(i) = shadow_val(i) + reHig*reWid/(L*K);
            end
        end

        % 修正系数
        shadow_val(i) = min(1, abs(1-shadow_val(i))*1.38);
    end

```



```

% 阴影效率
nsb = shadow_val;

% 镜面反射率
nref = 0.92*ones(1, length(points));

% 余弦效率
ncos = cos_val;

% 大气透射率
nair = air_val;

% 截断效率，若考虑为平行光可先忽略
ntrunc = tunc_val;

% 光学效率计算
nn = nsb.*nref.*ncos.*nair.*ntrunc;

% 输出热功率计算
Efield = DNI*sum(L*K*nn);

% aver_Efield = Efield/length(points);

res = [Efield, mean(nn)];
ssum = ssum + res(1);
ssum_n = ssum_n + res(2);
all_nn = all_nn + nn;
all_ncos = all_ncos + ncos;
end
end
ssum = ssum/4/5;
ssum_n = ssum_n/4/5;
all_nn = all_nn/4/5;
all_ncos = all_ncos/4/5;

ress = [ssum, ssum/(L*K*length(points))];
end

%-----
%-----
%-----

```

```

% problem3 主要支撑程序

clc;clear;

current_solution = struct();
current_solution.cen = [0,-185];
current_solution.L = linspace(8,4,100);
current_solution.K = current_solution.L*0.81;
current_solution.h = linspace(3.2,11,100);
current_solution.D = linspace(5,20,150);

current_point_set = generate_new3(current_solution.cen, current_solution.L, current_solution.K,
current_solution.h, current_solution.D);

lll = current_point_set.l_;
kkk = current_point_set.k_;
hhh = current_point_set.h_;
current_point_set = current_point_set.x_;

current_cost = objectiveFunction3(current_solution.cen(1), current_solution.cen(2), lll, kkk,
current_point_set);

current_cost = -current_cost(2);

% 随机扰动部分定日镜的参数并迭代寻优
max_iter = 1000;
for i = 1:max_iter
    for j = 1:10
        k = randi([1,50]);
        new_solution = current_solution;
        new_solution.L(k) = current_solution.L(k) + 0.05*randn();
        new_solution.K(k) = current_solution.K(k) + 0.05*randn();
        new_solution.h(k) = current_solution.h(k) + 0.05*randn();

        new_point_set = generate_new3(new_solution.cen, new_solution.L, new_solution.K,
new_solution.h, new_solution.D);

        lll = new_point_set.l_;
        kkk = new_point_set.k_;
        hhh = new_point_set.h_;
        new_point_set = new_point_set.x_;

        new_cost = objectiveFunction3(new_solution.cen(1), new_solution.cen(2), lll, kkk,

```

```

new_point_set);

    new_val = -new_cost(1);
    new_cost = -new_cost(2);

    if new_cost < current_cost && new_val < -60000;
        break;
    end
end
current_solution = new_solution;
current_point_set = new_point_set;
end

disp(1);

%-----
%-----
%-----

% 传入参数分别是中心塔坐标，定日镜长宽和定日镜中心坐标集合
function res = objectiveFunction3(cen_x, cen_y, L, K, xx)
    target = [cen_x cen_y 80];
    points = xx;

    %    for i = 1:length(points)
    % %        if points(i,2) < cen_y
    % %            L(i) = 5;
    % %            K(i) = 5;
    % %        end
    %        if sqrt(points(i,1)^2+points(i,2)^2) < 100
    %            L(i) = 8;
    %            K(i) = 8;
    %        end
    %        if points(i,2) < 350 && points(i,2) > 200 && points(i,1) < 50 && -50 < points(i,1)
    %            points(i,3) = (6-4.5)*(points(i,2)-200)/(350-200);
    %        end
    %    end

    % 阴影遮挡矩阵
    shadow = zeros(length(points), length(points));
    shadow_id = ones(1, length(points));
    for i = 1:length(points)
        for j = 1:length(points)

```

```

        if j == i
            continue;
        end
        if sqrt((points(j, 1)-points(i, 1))^2 + (points(j, 2)-points(i, 2))^2) < L(i)+10
            shadow(i, shadow_id(i)) = j;
            shadow_id(i) = shadow_id(i) + 1;
        end
    end
end

% 时间矩阵
dd = [0 92 184 275];
tt = [9 10.5 12 13.5 15];

ssum = 0;
ssum_n = 0;
all_nn = 0;
all_ncos = 0;
for d_id = 1:4
    for t_id = 1:5
        sun_angle = sun_direction(dd(d_id), tt(t_id));
        sun_vec = sun_vector(dd(d_id), tt(t_id));
        % rec = Coordinate2(points, L, K, sun_angle(1), sun_angle(2));
        norm_vec = NormalVector2(points, target, sun_angle(1), sun_angle(2));

        % 记录余弦效率的矩阵
        cos_val = zeros(1, length(points));

        % 记录阴影遮挡效率的矩阵
        shadow_val = zeros(1, length(points));

        % 记录截断效率的矩阵
        tunc_val = ones(1, length(points));

        % 记录大气透射率的矩阵
        air_val = zeros(1, length(points));

        % 计算 DNI
        temp_a = 0.4237-0.00821*(6-3)^2;
        temp_b = 0.5055+0.00595*(6.5-3)^2;
        temp_c = 0.2711+0.01858*(2.5-3)^2;
        DNI = 1.366*(temp_a+temp_b*exp(-temp_c/sin(sun_angle(1))));
    end
end

```

```

% 遍历每一个点，使用平面投影法计算阴影效率
for i = 1:length(points)
    a = sun_vec;
    n = norm_vec(i, :);

    % 计算余弦效率
    cos_val(i) = abs(dot(a, n)/norm(a)/norm(n));

    % 计算大气透射率
    air_val(i) = 0.99321-0.0001176*norm([0,0,80] - points(i,:)) + 1.97*1e-
8*norm(points(i,:) - [0,0,80])^2;

    [x,y,z] = deal(points(i,1), points(i,2), points(i,3));

    % 计算反射光线向量
    a_new = (a - dot(a, n/norm(n))*n/norm(n))- dot(a, n/norm(n))*n/norm(n);

    for j = 1:length(shadow(i, :))
        if shadow(i, j) == 0
            break;
        end
        % 判定阴影
        tmp_j = shadow(i, j);
        if dot(points(tmp_j, :) - points(i, :), [sun_vec(1), sun_vec(2), 0]) < 0
            [x0, y0, z0] = deal(points(tmp_j, 1), points(tmp_j, 2), points(tmp_j, 3));
            [n1, n2, n3] = deal(norm_vec(tmp_j, 1), norm_vec(tmp_j, 2),
norm_vec(tmp_j, 3));

            lambda = -(n1*(x-x0) + n2*(y-y0) + n3*(z-z0)) / (sun_vec(1)*n1 +
sun_vec(2)*n2 + sun_vec(3)*n3);
            point_new = [x+lambda*sun_vec(1), y+lambda*sun_vec(2),
z+lambda*sun_vec(3)];

            dis = norm(point_new-[x0,y0,z0]);
            hig = abs(point_new(3)-z0) * 1/(n1^2+n2^2);
            reHig = 1/2*(K(i)+K(tmp_j))-hig;
            reWid = 1/2*(L(i)+L(tmp_j))-sqrt(dis^2-hig^2);
            if reHig > 0 && reWid > 0
                shadow_val(i) = shadow_val(i) + reHig*reWid/(L(i)*K(i));
            end
        end

        % 判定遮挡
        [x0, y0, z0] = deal(points(tmp_j, 1), points(tmp_j, 2), points(tmp_j, 3));
        [n1, n2, n3] = deal(norm_vec(tmp_j, 1), norm_vec(tmp_j, 2), norm_vec(tmp_j,

```

```

3));

        lambda = -(n1*(x-x0) + n2*(y-y0) + n3*(z-z0)) / (a_new(1)*n1 + a_new(2)*n2 +
a_new(3)*n3);

        point_new = [x+lambda*a_new(1), y+lambda*a_new(2), z+lambda*a_new(3)];

        dis = norm(point_new-[x0,y0,z0]);
        hig = abs(point_new(3)-z0) * 1/(n1^2+n2^2);
        reHig = 1/2*(K(i)+K(tmp_j))-hig;
        reWid = 1/2*(L(i)+L(tmp_j))-sqrt(dis^2-hig^2);
        if reHig > 0 && reWid > 0
            shadow_val(i) = shadow_val(i) + reHig*reWid/(L(i)*K(i));
        end
    end

    % 修正系数
    shadow_val(i) = min(1, abs(1-shadow_val(i))*1.38);
end

% 阴影效率
nsb = shadow_val;

% 镜面反射率
nref = 0.92*ones(1, length(points));

% 余弦效率
ncos = cos_val;

% 大气透射率
nair = air_val;

% 截断效率, 若考虑为平行光可先忽略
ntrunc = tunc_val;

% 光学效率计算
nn = nsb.*nref.*ncos.*nair.*ntrunc;

% 输出热功率计算
Efield = DNI*sum(L.*K.*nn);

% aver_Efield = Efield/length(points);

res = [Efield, mean(nn)];
ssum = ssum + res(1);
ssum_n = ssum_n + res(2);

```

```
        all_nn = all_nn + nn;
        all_ncos = all_ncos + ncos;
    end

end

ssum = ssum/4/5;
ssum_n = ssum_n/4/5;
all_nn = all_nn/4/5;
all_ncos = all_ncos/4/5;

ress = [ssum, ssum/sum(L.*K)];
end
```